

Information Retrieval

Probabilistic Information Retrieval

Recap of the previous lectures

❖ Ranked retrieval and scoring

- Q: What are issues associated with Boolean retrieval that motivate ranked retrieval?
- Q: What are the common-sense assumptions of ranked retrieval?

❖ Vector space model (VSM)

- Q: What is TF-IDF weighting? How do we compute TF and how the IDF component?
- Q: Can we use raw term frequency as TF component? Why (not)?
- Q: What similarity/distance metrics do we employ in VSM?
- Q: Compare cosine similarity/distance and Euclidean distance

❖ Optimizing VSM retrieval

- Q: How can we use an inverted index to speed up VSM retrieval?
- Q: What is a tiered index?

Recap of the previous lectures

- ❖ IR models for ranked retrieval
 - Produce the ordering over the documents in the collection
- ❖ Assumptions of VSM model:
 - Term more relevant the more frequent it is in the document (TF component)
 - Term more informative the less frequent it is among documents (IDF component)
- ❖ Document d_j is represented by term vector $[w_{1j}, w_{2j}, \dots, w_{tj}]$ where each weight is the TF-IDF score of the term t_i and document d_j
- ❖ Ranking function r for VSM: cosine similarity between TF-IDF vectors of query and document
- ❖ Now, we examine what the ranking function r looks like for the probabilistic models for ranked retrieval
 - Binary probabilistic model
 - Extensions that additionally take into account term frequency

Outline

- ❖ Probabilistic Ranking Principle (PRP)
- ❖ Basics of probability theory
- ❖ Probabilistic ranking (log-odds)
- ❖ Binary Independence Model (BIM)
- ❖ BIM Extensions
 - Two-Poisson model
 - BM11
 - BM25

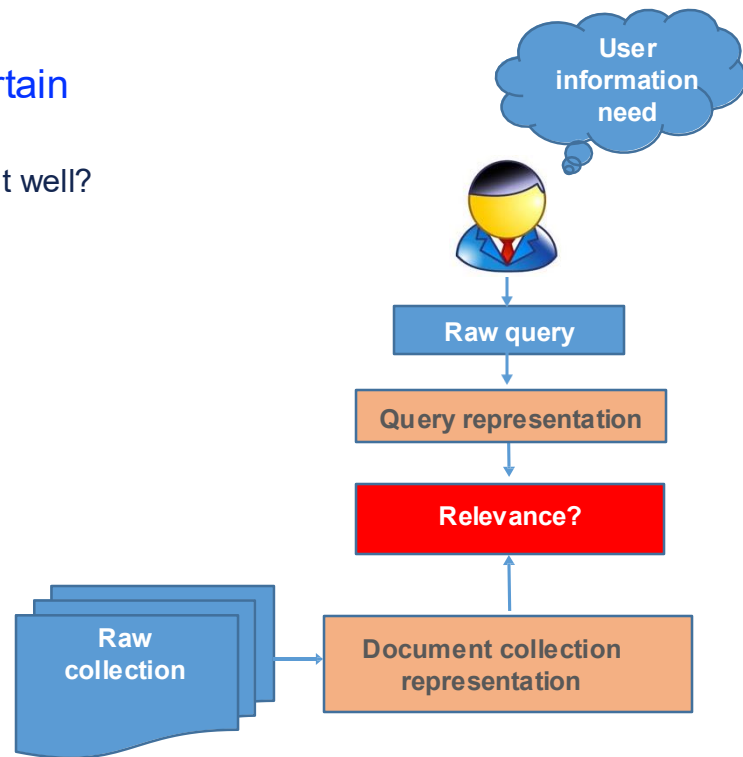
Why probability theory in IR?

❖ As a process, retrieval is inherently uncertain

- Understanding of user's information needs is uncertain
 - Are we sure the user mapped his need into a good query?
 - Even if the query represents well the need, did we represent it well?
- Estimating document relevance for the query
 - Uncertainty from selection of document representation
 - Uncertainty from matching query and documents

❖ Probability theory is a common framework for modelling uncertainty

- Idea: Rank by the probability of relevance of the document w.r.t. information need
 - $P(R=1|\text{document, query})$



Basic Idea

Query	Doc	Relevance
q1	d1	1
q1	d2	0
q1	d3	1
q1	d1	0
q1	d2	0
q2	d1	1
q2	d2	0
q3	d1	1
q4	d2	0

Basic Idea

Query	Doc	Relevance
q1	d1	1
q1	d2	0
q1	d3	1
q1	d1	0
q1	d2	0
q2	d1	1
q2	d2	0
q3	d1	1
q4	d2	0

$$\begin{aligned}\text{Score}(d,q) &= p(R=1|d,q) \\ &= \text{count}(q,d,R=1) / \text{count}(q,d)\end{aligned}$$

$$P(R=1|q1,d1) = 1/2$$

$$P(R=1|q1,d2) = 0/2$$

$$P(R=1|q1,d3) = 1/1$$

Probabilistic approach to retrieval

- ❖ An IR system is primarily uncertain about
 - Understanding of the query
 - Whether a document satisfies the query
- ❖ Probability theory
 - Provides the foundation for reasoning under uncertainty
 - Probabilistic information retrieval models estimate how likely it is that a document is relevant for a query
- ❖ Probabilistic IR models
 - Classic probabilistic models (BIM, Two Poisson, BM11, BM25)
 - Language modelling for IR (next lectures)
- ❖ Probabilistic IR models are among the best-performing and most widely used IR models

Probabilistic Ranking Principle

- ❖ Assume the ranked retrieval setting
 - We are given a query q and a document collection D
 - Ordered list of documents from D is to be returned for q
- ❖ We model relevance (non-relevance) as **random binary variables**
 - $R_{d,q} = 1$ if document d from D is relevant for query q ,
 - $R_{d,q} = 0$ otherwise
- ❖ **Probabilistic ranking principle:** The information retrieval system will reach the best obtainable efficiency if the documents are ranked decreasingly according to their probability of relevance
 - I.e., decreasingly in terms of $P(R_{d,q} = 1)$, or, equivalently, $P(R = 1 \mid d, q)$

Probabilistic ranking principle

Original explanation of probabilistic ranking principle (Robertson, 1977):

If [the IR] system's response to each [query] is a ranking of the documents [...] in order of decreasing probability of relevance to the [query], **where the probabilities are estimated as accurately as possible on the basis of whatever data have been made available to the system for this purpose**, the overall effectiveness of the system to its user will be the best **that is obtainable on the basis of those data**

- ❖ Probabilistic retrieval models aim to answer the following question: "What is the probability that the user will judge this document as relevant for this query?"
 - Compute the best estimate from the available data (query and document collection)
 - How do we formalize this question?

Formalization of the prob. ranking principle

❖ We introduce **sets of random variables** for **terms** in query and documents:

1. $D = \{D_1, D_2, \dots, D_N\}$

- Set of random variables representing terms of documents

2. $Q = \{Q_1, Q_2, \dots, Q_L\}$

- Set of random variables representing terms of the query

❖ We introduce a random variable representing user's **relevance judgement** for a concrete query-document pair

3. $R \in \{0, 1\}$

- $R = 1$ if D is relevant for Q , $R = 0$ otherwise

Formalization of the prob. ranking principle

- ❖ PRP's central question:

- What is the probability that the user will judge this document as relevant for this query?

- ❖ Random variables

- ❖ $D = \{D_1, D_2, \dots, D_N\}; Q = \{Q_1, Q_2, \dots, Q_L\}; R \in \{0, 1\}$

- ❖ The above question for a query q and a document d is now equivalent to estimating the probability:

$$P(R = 1 \mid D = d, Q = q)$$

- ❖ We will examine different ways of estimating this probability

Basics of probability theory

- ❖ Atomic events are represented with random variables – $A, B, \dots, Z$ – taking one of the possible values
 - a probability assigned to every event, e.g., $P(A = a)$
- ❖ For two events A and B
 - **Joint probability** $P(A, B)$ is the probability of both events occurring. If events are independent, $P(A, B) = P(A) * P(B)$
 - **Conditional probability** $P(A | B)$ is the probability of event A occurring given the previous occurrence of event B
 - **Chain rule** allows to write the joint probability using conditional probabilities
$$P(A, B, C) = P(A) * P(B | A) * P(C | A, B)$$
 - **Partition rule** – if one of the events can be divided into disjoint subcases, its probability is the sum of the probabilities of the subcases
$$P(B) = P(A, B) + P(\text{not } A, B)$$
$$P(B) = P(A = a_1, B) + P(A = a_2, B) + \dots + P(A = a_N, B)$$

Basics of probability theory

- ❖ **Bayes' Rule** inverts the conditional probabilities:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)} = \left[\frac{P(B|A)}{\sum_{X \in \{A, \bar{A}\}} P(B|X)P(X)} \right] P(A)$$

- ❖ Can be thought of as a way of **updating probabilities**:
 - Start off with **prior probability** $P(A)$ (initial estimate of how likely event A is in the absence of any other information)
 - Derive a **posterior probability** $P(A|B)$ after having seen the evidence B, based on the likelihood of B occurring in the two cases that A does or does not hold

Basics of probability theory

- ❖ **Odds** of an event occurring is a ratio of the probability of event occurring and it not occurring (multiplier for how probability changes)

$$O(A) = \frac{P(A)}{P(\bar{A})} = \frac{P(A)}{1 - P(A)}$$

- ❖ Often, instead of raw odds, we compute the logarithm of the odds, **log-odds** (for numeric convenience)

$$\log(O(A)) = \log P(A) - \log(1 - P(A))$$

Binary Independence Model

❖ Traditionally used in conjunction with PRP with the following assumptions:

– “Binary” (equivalent to Boolean): documents are represented as binary incidence vectors of terms

- e.g., document d represented by vector $x = (x_1, \dots, x_M)$, where $x_t = 1$ if term t occurs in d and $x_t = 0$ otherwise
- Different documents can be modelled as the same vector

	Anthony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	...
ANTHONY	1	1	0	0	0	1	
BRUTUS	1	1	0	1	0	0	
CAESAR	1	1	0	1	1	1	
CALPURNIA	0	1	0	0	0	0	
CLEOPATRA	1	0	0	0	0	0	
MERCY	1	0	1	1	1	1	
WORSER	1	0	1	1	1	0	
...							

Each document is represented as a **binary vector** $\in \{0, 1\}^M$.

– “Independence”: the presence or absence of a word in a document is independent of the presence or absence of any other word

- not true, but works in practice

Binary Independence Model

- ❖ To make a probabilistic retrieval strategy precise, we need to estimate how terms or other variables in documents contribute to relevance.
- ❖ Find measurable statistics that affect judgments about document relevance.
 - e.g., term frequency, document frequency, document length, etc.
- ❖ Combine these statistics to estimate the probability $P(R|d, q)$ of document relevance.

Binary Independence Model

- ❖ Given query q ,
 - for each document d need to compute $p(R|q,d)$
 - replace with computing $p(R|q,x)$ where x is binary term incidence vector representing d
 - Interested only in ranking
- ❖ Will use odds and Bayes' Rule:
 - Odds = the probability that the event will occur divided by the probability that the event will not occur, i.e., $p/(1-p)$.

$$O(R | q, x) = \frac{p(R=1 | q, x)}{p(R=0 | q, x)} = \frac{\frac{p(R=1 | q)p(x | R=1, q)}{p(x | q)}}{\frac{p(R=0 | q)p(x | R=0, q)}{p(x | q)}}$$

Binary Independence Model

$$O(R | q, x) = \frac{p(R=1 | q, x)}{p(R=0 | q, x)} = \frac{p(R=1 | q)}{p(R=0 | q)} \frac{p(x | R=1, q)}{p(x | R=0, q)}$$

Constant for a given query, can be ignored

Needs estimation

- Using Independence Assumption:

$$O(R | q, x) = O(R | q) \prod_{i=1}^n \frac{p(x_i | R=1, q)}{p(x_i | R=0, q)}$$

- Since x_i is either 0 or 1, we can separate the terms:

$$O(R | q, x) = O(R | q) \prod_{x_i=1} \frac{p(x_i=1 | R=1, q)}{p(x_i=1 | R=0, q)} \prod_{x_i=0} \frac{p(x_i=0 | R=1, q)}{p(x_i=0 | R=0, q)}$$

Binary Independence Model

- ❖ Let p_i be the probability of a term appearing in a **relevant** document

$$p_i = p(x_i = 1 \mid R = 1, q);$$

- ❖ And the probability of a term appearing in a **nonrelevant** document.

$$r_i = p(x_i = 1 \mid R = 0, q);$$

- ❖ Assume, that all terms not occurring in the query they are equally likely to occur in relevant and nonrelevant documents

$$p_i = r_i$$

- ❖ We only consider terms in the products that appear in the query:

$$O(R \mid q, x) = O(R \mid q) \prod_{\substack{x_i=1 \\ q_i=1}} \frac{p_i}{r_i} \prod_{\substack{x_i=0 \\ q_i=1}} \frac{(1-p_i)}{(1-r_i)}$$

Terms in document Terms not in document

Binary Independence Model

$$O(R | q, x) = O(R | q) \prod_{\substack{x_i=q_i=1 \\ q_i=1}} \frac{p_i}{r_i} \prod_{\substack{x_i=0 \\ q_i=1}} \frac{1-p_i}{1-r_i}$$

$$O(R | q, x) = O(R | q) \prod_{\substack{x_i=1 \\ q_i=1}} \frac{p_i}{r_i} \prod_{\substack{x_i=1 \\ q_i=1}} \frac{1-r_i}{1-p_i} \prod_{\substack{x_i=0 \\ q_i=1}} \frac{1-p_i}{1-r_i}$$

$$O(R | q, x) = O(R | q) \prod_{\substack{x_i=q_i=1 \\ q_i=1}} \frac{p_i(1-r_i)}{r_i(1-p_i)} \prod_{q_i=1} \frac{1-p_i}{1-r_i}$$

All matching terms

All query terms

Binary Independence Model

$$O(R | q, \vec{x}) = \boxed{O(R | q)} \cdot \boxed{\prod_{x_i=q_i=1} \frac{p_i(1-r_i)}{r_i(1-p_i)}} \cdot \boxed{\prod_{q_i=1} \frac{1-p_i}{1-r_i}}$$

Constant for each query

↑
The only quantity to be estimated for rankings

- ❖ We can equally rank documents by the logarithm since log is a monotonic function.

– Hence the Retrieval Status Value (RSV) in this model:

$$RSV = \log \prod_{x_i=q_i=1} \frac{p_i(1-r_i)}{r_i(1-p_i)} = \sum_{x_i=q_i=1} \log \frac{p_i(1-r_i)}{r_i(1-p_i)}$$

Binary Independence Model

- ❖ Equivalent: rank documents using log odds ratios for query terms c_i :

$$RSV = \sum_{x_i=q_i=1} c_i; \quad c_i = \log \frac{p_i(1-r_i)}{r_i(1-p_i)}$$

$p_i/(1-p_i)$ odds of the term appearing if the document is relevant

$r_i/(1-r_i)$ odds of the term appearing if the document is not relevant

• or $\log(p_i/(1-p_i)) - \log(r_i/(1-r_i))$

- $c_i = 0$: term has equal odds of appearing in relevant and nonrel. docs.
- c_i positive: higher odds to appear in relevant documents.
- c_i negative: higher odds to appear in nonrelevant documents.
- ❖ So BIM and vector space model are identical on an operational level
 - ... except that the term weights are different.
- ❖ In particular, we can utilise the same data structures (such as inverted indexes) for both models.

Binary Independence Model

❖ Estimating RSV coefficients in theory

$$c_i = \log \frac{p_i(1-r_i)}{r_i(1-p_i)}$$

Documents	Relevant	Non-Relevant
<i>Term present</i> $x_i=1$	p_i	r_i
<i>Term not present</i> $x_i=0$	$1-p_i$	$1-r_i$

$$p_i = p(x_i = 1 \mid R = 1, q); \quad r_i = p(x_i = 1 \mid R = 0, q);$$

Binary Independence Model

- ❖ Estimating RSV coefficients in theory
- ❖ Consider the contingency table for each term i

$$c_i = \log \frac{p_i(1-r_i)}{r_i(1-p_i)}$$

Documents	Relevant	Non-Relevant	Total
<i>Term present</i> $x_i=1$	s	$n-s$	n
<i>Term not present</i> $x_i=0$	$S-s$	$N-n-S+s$	$N-n$
Total	S	$N-S$	N

$$p_i = p(x_i = 1 \mid R = 1, q); \quad r_i = p(x_i = 1 \mid R = 0, q);$$

Smoothing
Add 0.5 to
each term to
avoid zeroes

Binary Independence Model

❖ Estimating RSV coefficients in theory

$$c_i = \log \frac{p_i(1-r_i)}{r_i(1-p_i)}$$

❖ Consider the contingency table for each term i

Documents	Relevant	Non-Relevant	Total
<i>Term present</i> $x_i=1$	s	$n-s$	n
<i>Term not present</i> $x_i=0$	$S-s$	$N-n-S+s$	$N-n$
Total	S	$N-S$	N

$$p_i \approx \frac{s}{S} \quad r_i \approx \frac{(n-s)}{(N-S)}$$

$$c_i \approx K(N, n, S, s) = \log \frac{s/(S-s)}{(n-s)/(N-n-S+s)}$$

Smoothing
Add 0.5 to
each term to
avoid zeroes

Key challenge: estimation

$$c_i = \log \frac{p_i(1-r_i)}{r_i(1-p_i)}$$

- ❖ If non-relevant documents are approximated by the whole collection, then r_i (prob. of occurrence in non-relevant documents for query) is n/N (i.e., df_i/N) and

$$\log \frac{1-r_i}{r_i} = \log \frac{N-n-S+s}{n-s} \square \log \frac{N-n}{n} \square \log \frac{N}{n} = IDF!$$

Key challenge: estimation

- ❖ p_i (probability of occurrence in relevant documents) cannot be approximated as easily
- ❖ p_i can be estimated in various ways:
 - from relevant documents if you know some
 - Relevance weighting can be used in a feedback loop
 - constant (Croft and Harper combination match model) –with $p_i=0.5$, this is simply idf weighting of query terms

$$RSV = \prod_{x_i=q_i=1} \log \frac{N}{n_i}$$

- proportional to prob. of occurrence in collection
 - E.g., $1/3 + 2/3 \text{ df}_i/N$

Probabilistic Relevance Feedback

- ❖ Guess a preliminary probabilistic model of $R=1$ documents; use it to retrieve a set of documents
- ❖ Interact with the user to refine the model:
partition subset V into V_R and V_{NR}
- ❖ Re-estimate p_i and r_i on the basis of these
 - If i appears in V_{Ri} within set of documents V_R : $p_i = |V_{Ri}|/|V_R|$
 - Or combine new information with previous guess (Bayesian update):

$$p_i^{(n+1)} = \frac{|V_{Ri}| + \kappa p_i^n}{|V_R| + \kappa}$$

p_i – Bayesian prior
 κ – prior weight

- ❖ Repeat, thus generating a succession of approximations to relevant documents

Iteratively estimating p_i and r_i (= Pseudo-relevance feedback)

1. Assume that p_i is constant over all x_i in query and r_i as before
 - $p_i = 0.5$ (even odds) for any given doc
2. Determine guess of relevant document set:
 - V is fixed size set of highest ranked documents on this model
3. We need to improve our guesses for p_i and r_i , so
 - Use distribution of x_i in docs in V . Let V_i be set of documents containing x_i
 - $p_i = |V_i| / |V|$
 - Assume if not retrieved then not relevant
 - $r_i = (n_i - |V_i|) / (N - |V|)$
4. Go to 2. until converges then return ranking

PRP and BIM

- ❖ BIM requires restrictive assumptions:
 - Term independence
 - Terms not in query don't affect the outcome
 - Boolean representation of documents/queries/relevance
 - Document relevance values are independent
- ❖ Problem: either requires partial relevance information or, seemingly, it can only derive somewhat inferior term weights

A key limitation of the BIM

- ❖ BIM – like much of original IR – was designed for titles or abstracts, and not for modern full text search
- ❖ We want to pay attention to term frequency and document lengths, just like in other models we discuss
- ❖ Want some model of how often terms occur in docs

$$c_i = \log \frac{p_{tf} r_0}{p_0 r_{tf}}$$

How different are vector space model and BIM?

- ❖ They are not that different.
- ❖ In either case, we build an information retrieval scheme in the exact same way.
- ❖ For probabilistic IR, at the end, we score queries not by cosine similarity and tf-idf in a vector space, but by a slightly different formula motivated by probability theory.
- ❖ Next: how to add term frequency and length normalization to the probabilistic model?

BIM Extensions

❖ Two-Poisson model

❖ BM11

❖ BM25

Two Poisson model

- ❖ All BIM extensions introduce the **additional scaling** of term weights w_t
 - We move away from binary representation – account for **term frequencies**
- ❖ **Two Poisson model** models frequencies with the Poisson distribution
 - Implicit assumption: all documents are of **equal length**

$$rel(D, Q) = \sum_{t \in Q} \frac{f_{t,D}(k+1)}{f_{t,D} + k} \cdot w_t$$

- $f_{t,D}$ is the raw frequency of term t in document D
 - k is a real constant, usually $1 \leq k < 2$
- ❖ **Effect**: weights of higher frequency words get boosted

BM11

- ❖ By using **raw term frequency**, **Two Poisson model** assumes all documents are equally long – but this is a **faulty assumption**
- ❖ **BM11 model** corrects the weight scaling factor of **Two Poisson model** to account for different document lengths
 - l_{avg} – average length of documents in the collection
 - l_d – the length of the document D

$$rel(D, Q) = \sum_{t \in Q} \frac{f_{t,D}(k+1)}{f_{t,D} + k \frac{l_d}{l_{avg}}} \cdot w_t$$

- ❖ **Effect**: raw word frequencies **are dampened/boosted** depending on the **above/below** average document length

BM25 (BestMatch25 or Okapi)

- ❖ While BM11 removes the assumption of equal document length, in practice, it has problems
 - Long relevant documents are getting too much dampening
 - Short irrelevant documents are getting too much boosting
- ❖ To control the amount of correction (dampening/boosting) for document length, the BM25 model introduces an additional parameter b

$$rel(D, Q) = \sum_{t \in Q} \frac{f_{t,D}(k+1)}{f_{t,D} + k \frac{l_d}{l_{avg}} b + k(1-b)} \cdot w_t$$

- ❖ The most common value for parameter b is 0.75
- ❖ BM25 is the most famous probabilistic ranking function
 - Many IR systems use BM25 as the primary ranking function

Probabilistic models vs. VSM

- ❖ Different theoretical underpinnings, but similar ranking effects
 - The ranking function of the probabilistic models is grounded in **probability theory**
 - The ranking function of VSM – cosine similarity – is grounded in **vector algebra**
- ❖ Binary independence model – **binary term weights**
 - Similar effect to **ignoring the TF component** in VSM (i.e., just IDF weighting)
- ❖ Two-Poisson model – **raw term frequency**
 - Similar effect to using **raw frequency as TF** component in VSM
- ❖ BM11 – accounts for **document length**
 - Similar effect to using **length-normalised TF** component in VSM ($f_{t,D} / \max f_{t',D}$)
- ❖ BM25 – **dampens** the effects of **document length**
 - Similar to taking a **logarithm of length-normalised frequency** as TF in VSM ($\log(f_{t,D} / \max f_{t',D})$)

Why is BM25 better than VSM tf-idf?

- ❖ Suppose your query is [machine learning]
- ❖ Suppose you have 2 documents with term counts:

- doc1: learning 1024; machine 1
- doc2: learning 16; machine 8

- ❖ tf-idf: $(1 + \log_2 \text{tf}) * \log_2 (N/\text{df})$

- doc1: $11 * 7 + 1 * 10 = 87$
- doc2: $5 * 7 + 4 * 10 = 75$

- ❖ BM25: $k_1 = 2$

- doc1: $7 * 3 + 10 * 1 = 31$
- doc2: $7 * 2.67 + 10 * 2.4 = 42.7$

$$RSV^{BM25} = \prod_{i \in q} \log \frac{N}{df_i} \prod \frac{(k_1 + 1)tf_i}{k_1((1-b) + b \frac{dl}{avdl}) + tf_i}$$